



Git Basics & Tooling

A Beginner's Lab for Version Control

Estimated Time: 45–60 minutes | Level: Beginner

What You Will Master

By the end of this lab, you will understand and execute the industry-standard version control workflow.



Install and configure local Git environments



Establish a GitHub account and remote access



Clone, edit, and track files locally



Execute the core workflow:
Add → Commit → Push

Required Tooling



Git

The core version control engine. Runs locally on your machine.



Visual Studio Code

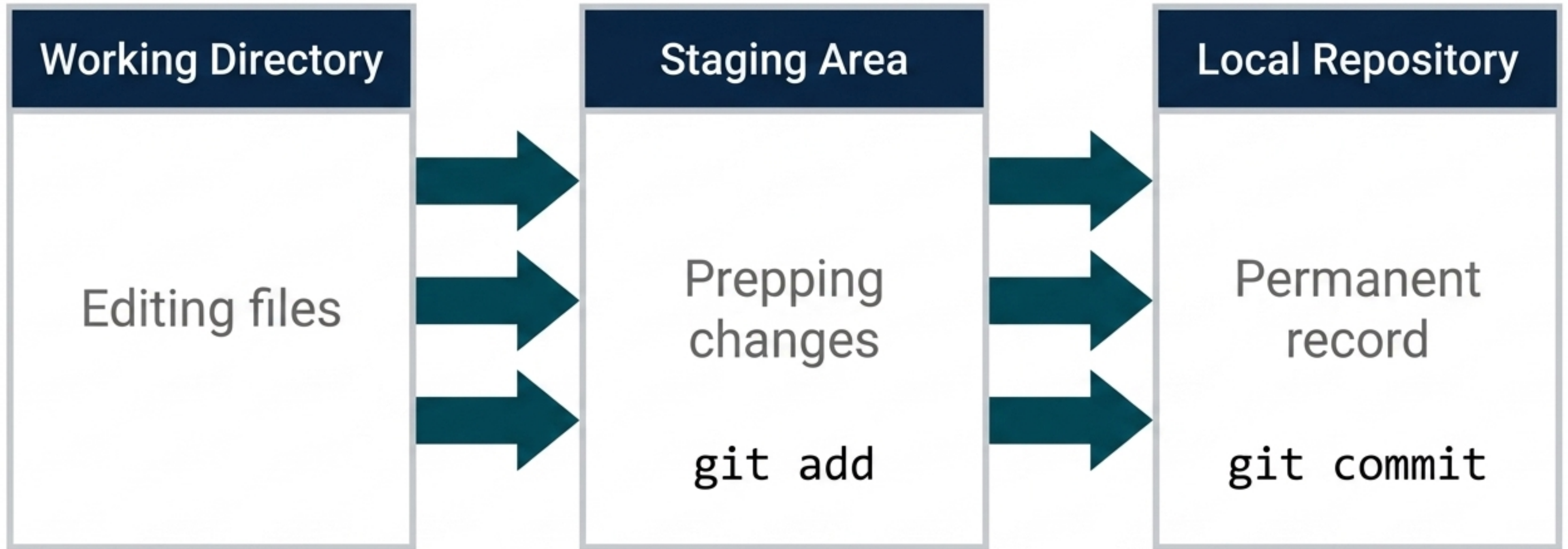
Recommended, industry-standard text editor for modifying files.



GitHub Account

Cloud hosting platform for your remote repositories.

The Three Stages of Git



Git does not track changes automatically.
You must explicitly move files through these three stages.

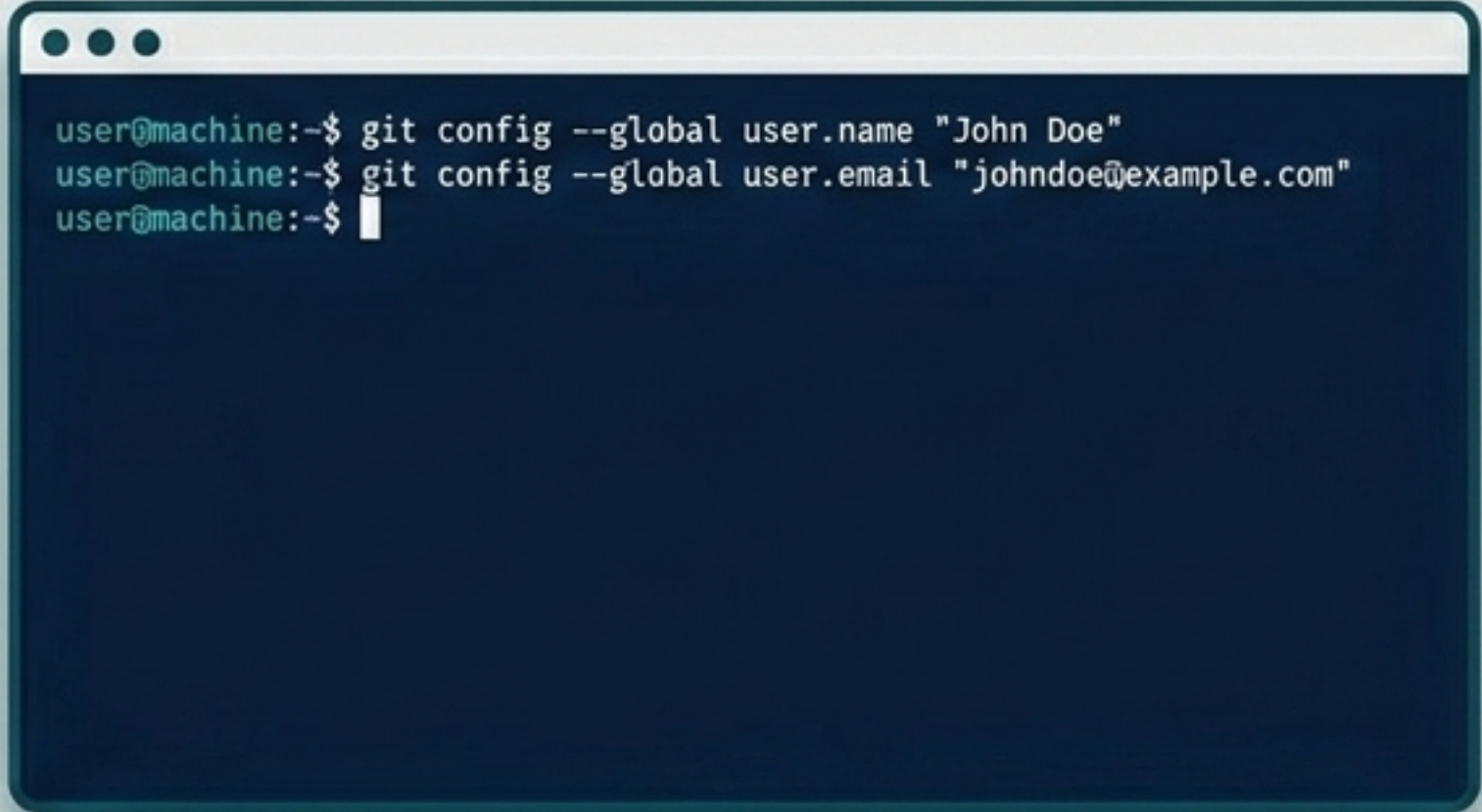
Establish Your Identity

git config

Git bakes your identity into every commit. This configuration runs once per machine to set global variables.

```
git config --global user.name "John Doe"
```

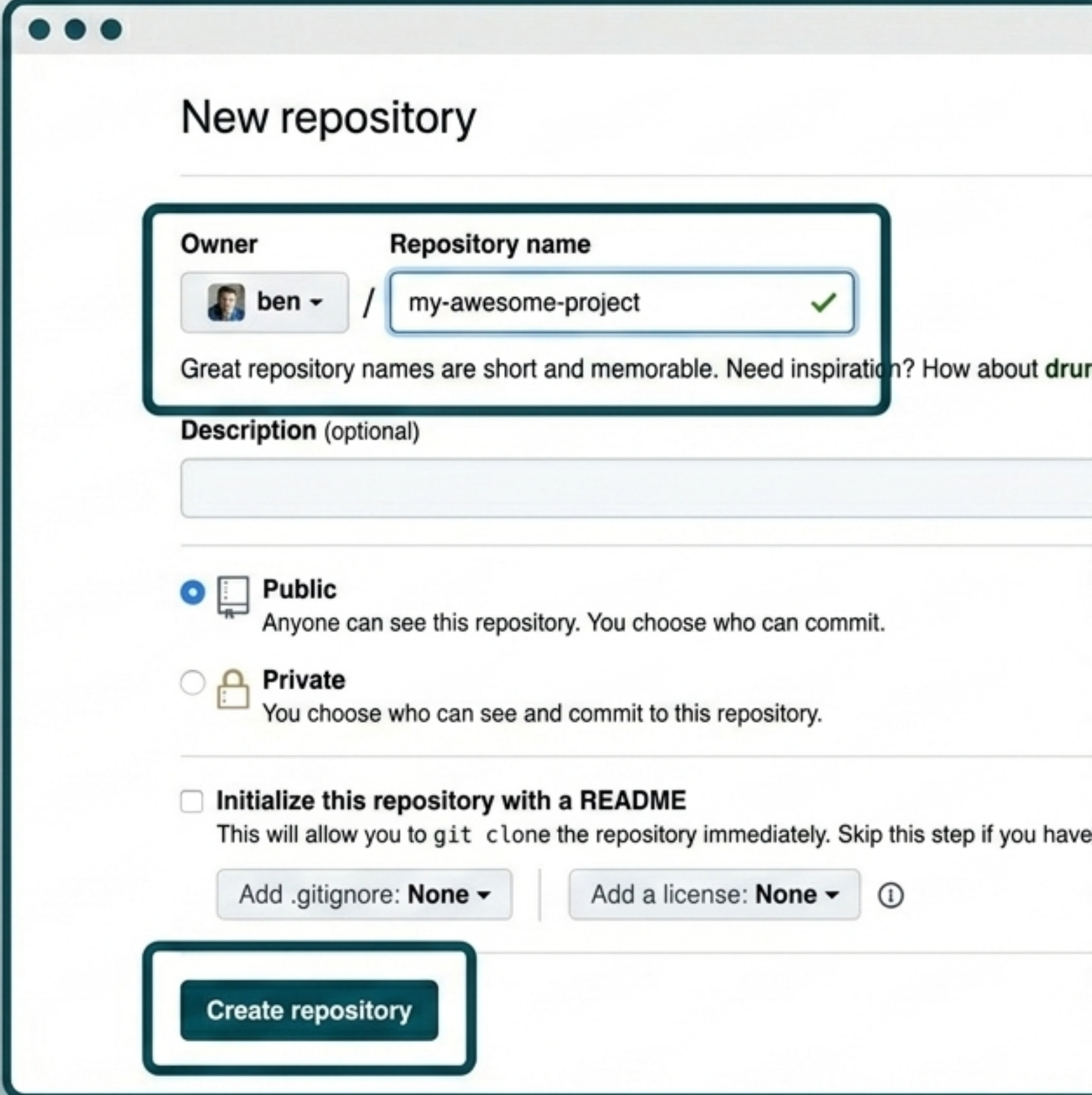
```
git config --global user.email "johndoe@example.com"
```

A terminal window with a dark blue background and white text. It shows three lines of commands being entered at a prompt 'user@machine:~\$'. The first line is 'git config --global user.name "John Doe"', the second is 'git config --global user.email "johndoe@example.com"', and the third is an empty line with the prompt 'user@machine:~\$' and a cursor.

```
user@machine:~$ git config --global user.name "John Doe"
user@machine:~$ git config --global user.email "johndoe@example.com"
user@machine:~$
```

Initialize Remote Repository

1. Log into GitHub.com
2. Click the + icon → New repository
3. Name your project
4. Keep it Public or Private
5. Click Create repository



The screenshot shows the GitHub 'New repository' page. A dark teal border highlights the 'Owner' and 'Repository name' fields, which are filled with 'ben' and 'my-awesome-project' respectively, with a green checkmark next to the repository name. Below this, the 'Description (optional)' field is empty. The 'Public' option is selected with a radio button, and the 'Private' option is unselected. The 'Initialize this repository with a README' checkbox is also unselected. At the bottom, the 'Create repository' button is highlighted with a dark teal border. The page title is 'New repository'.

New repository

Owner:  ben / Repository name: my-awesome-project ✓

Great repository names are short and memorable. Need inspiration? How about [drum](#)

Description (optional)

☒ **Public**
Anyone can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

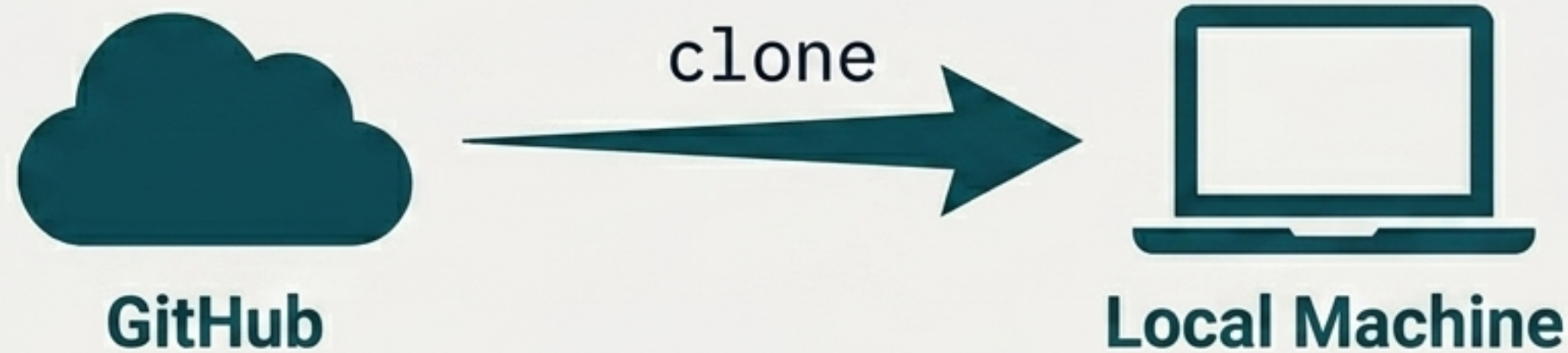
☐ **Initialize this repository with a README**
This will allow you to `git clone` the repository immediately. Skip this step if you have

Add .gitignore: **None** | Add a license: **None** ⓘ

Create repository

Clone to Local Machine `git clone`

Cloning downloads a full copy of the remote repository to your local system, ready for editing.



```
git clone https://github.com/[username]/[repo].git
```

➤ **Action:** Open the newly created folder in Visual Studio Code to begin working.

Edit & Stage `git add`

1. Create a new file
(e.g., README.md)
2. Check status:
`git status`
3. Stage the file:
`git add README.md`

```
$ git add README.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README.md
```

Commit `git commit`

Committing permanently records your staged file snapshots in the local version history.

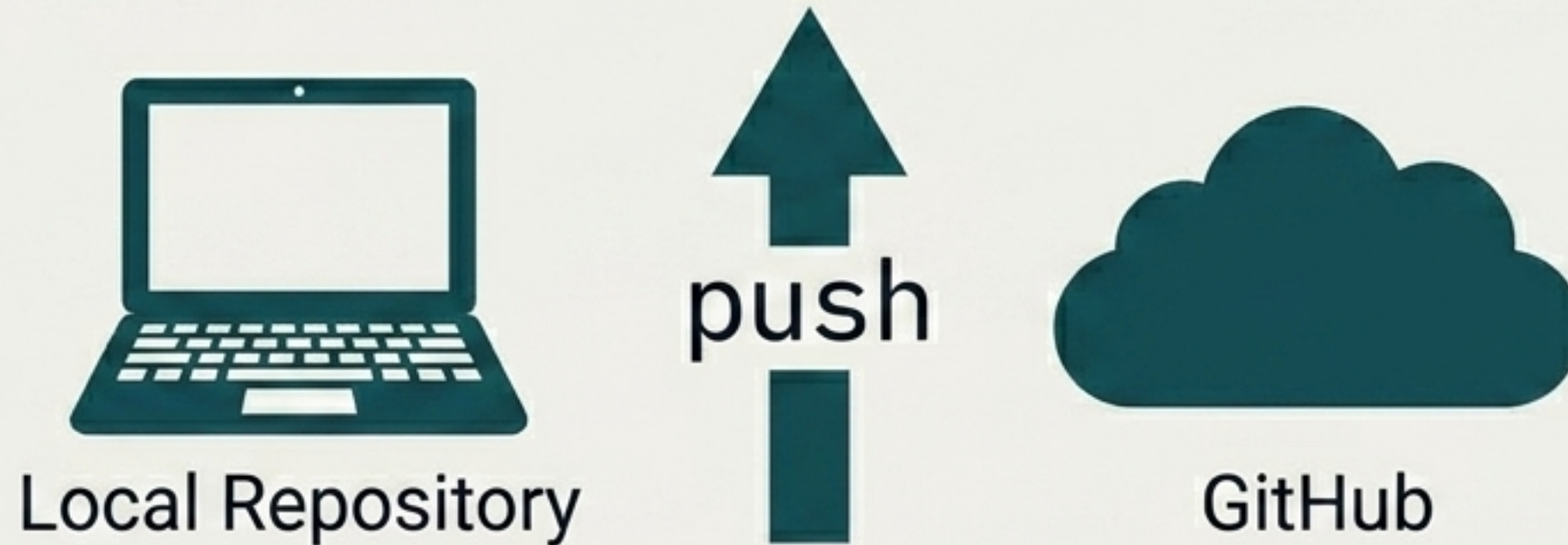


```
git commit -m "Initial project version"
```

BEST PRACTICE: Always write clear, descriptive commit messages. Explain *what* was changed and *why*.

Publish to Remote `git push`

Upload all local commits to the corresponding remote branch on GitHub to share your work.



```
git push origin main
```

Note: If prompted for authentication during this step, refer to the next slide for credential management.

Handling Authentication

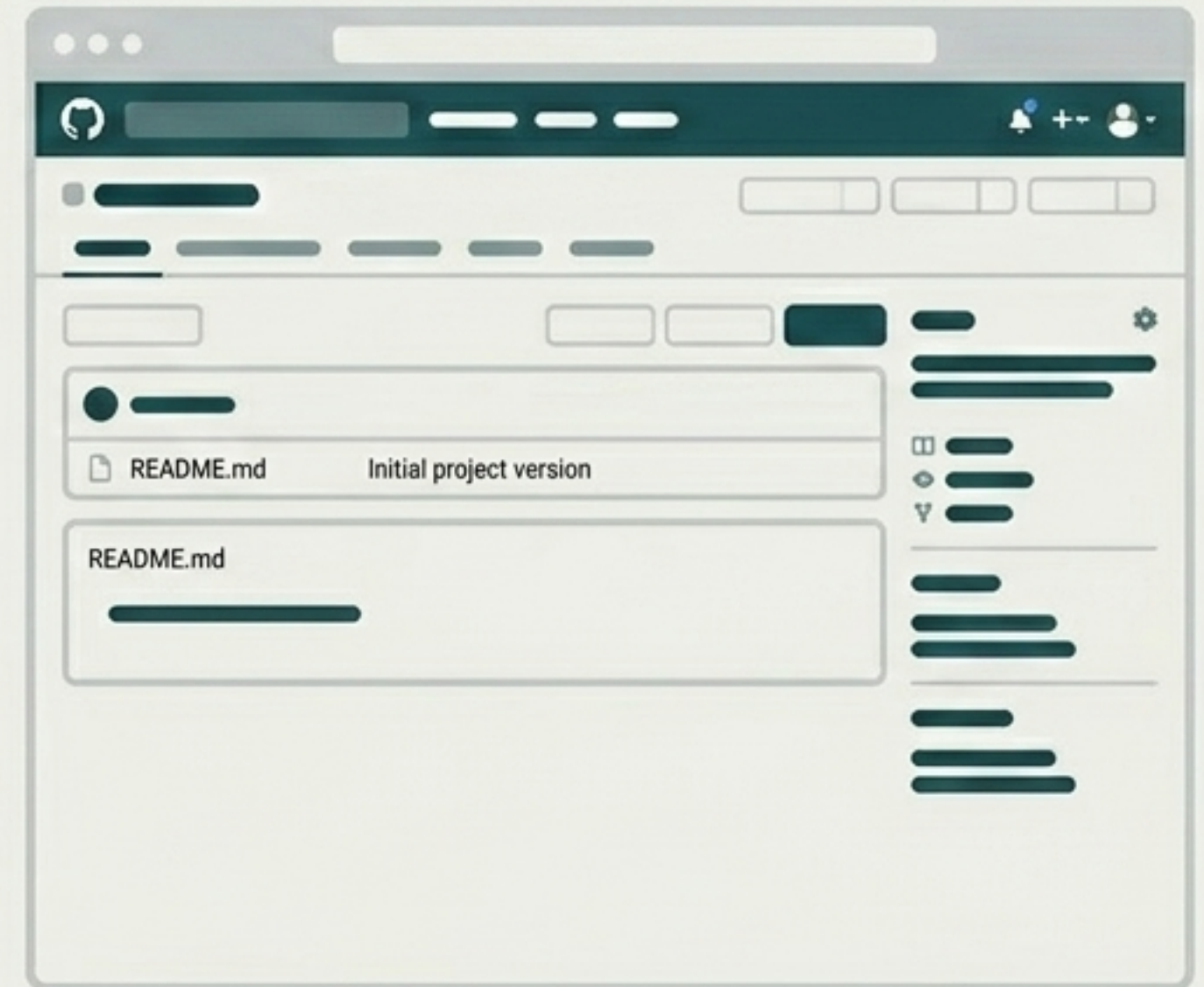
GitHub killed password authentication for Git operations. You must use one of two **secure methods** to push code:

	Personal Access Token (PAT) 	Git Credential Manager (GCM) 
Setup	Manual creation via GitHub Developer Settings	Automatic OS integration
Security	Granular scopes (read/write), expires automatically	OS-level keychain encryption
Convenience	Requires copy/pasting token	Set it and forget it seamless operation

Lab Deliverables & Verification

Verify your success by checking your GitHub repository online.

- [] Git is configured locally (Identity set)
- [] Remote repository exists on GitHub
- [] Local repository successfully cloned
- [] At least one file edited, staged, and committed
- [] Changes successfully pushed and visible on GitHub.com



Common Issues & Quick Fixes

Issue: Authentication Failed

Fix: Passwords are deprecated. Generate a Personal Access Token (PAT) or install Git Credential Manager.

Issue: Cannot Push (Rejected)

Fix: The remote contains changes you don't have locally. Run ``git pull`` before ``git push``.

Issue: Committed to the Wrong Branch

Fix: Do not panic. Use ``git status`` to check your location, and ``git checkout`` to change branches.

Bonus Challenges (Optional)



The Ignore List

Create a `.gitignore` file to exclude specific files (like system logs or secrets) from being tracked.



Remote Editing

Make a change to a file directly on GitHub.com, then sync it to your local machine using the `'git pull'` command.



Time Travel

Use the `'git log'` command in your terminal to view the permanent commit history of your local repository.

Key Takeaways

Git $\equiv$ **Local Engine** **GitHub** $\equiv$ **Cloud Host**

Local Loop: **Edit** $\rightarrow$ **Add** $\rightarrow$ **Commit**

Sync Loop: **Pull** $\rightarrow$ **Push**

When in doubt, type ``git status``.

You are now ready to version control your academic and professional projects.